

DS Project Map — Customer Segmentation MLOps

STP-CS-2026 | Otto-von-Guericke-Universität Magdeburg | Summer Semester 2026

Supervisors: Prof. Dr. Klaus Turowski · M. Sc. Christian Haertel

Repository: <https://github.com/Thanderer/stp-customer-segmentation>

MLflow/DagsHub: <https://dagshub.com/Thanderer/stp-customer-segmentation>

Process Model: Haertel et al. (2022) — MLOps-based DSPM Reference: Kreuzberger et al. (2023)
— IEEE Access DOI: 10.1109/ACCESS.2023.3262138

1. Team & Roles

Member	DSPM Role Category	Key Responsibilities
Kirtan	Strategy & PM · Domain Expertise	Business objectives, documentation, project plan, cluster labelling, evaluation reports, final report
Ayush	Data Management & Analysis · Analytical Modeling	EDA, RFM engineering, model training, MLflow experiments, model assessment, drift simulation
Jash	Data Infrastructure & Operations	Repo setup, data pipeline, MLflow server, model registry, FastAPI, Docker, CI/CD, monitoring dashboards

2. Business Understanding

2.1 Business Problem

A UK-based online retailer communicates with all customers using identical marketing messages — the same emails, discounts, and newsletters regardless of individual purchase behaviour. This results in:

- Low campaign engagement from customers receiving irrelevant messages
- Wasted marketing spend on inactive or low-value customers
- Missed revenue from high-value customers who receive no preferential treatment
- No mechanism to identify and act on customers who are at risk of churning

2.2 Business Goals

Primary Goal - Enable personalised, segment-specific marketing by grouping customers according to purchase behaviour.

Goal	Measurement Proxy	How Assessed in This Project
Reduce customer churn by identifying At-Risk customers early and enabling targeted re-engagement campaigns	Percentage of At-Risk customers identified per period	Cluster profiling — At-Risk segment defined by high Recency, low Frequency
Increase campaign engagement by replacing mass emails with segment-specific messaging	Intended engagement uplift per segment	Validated by marketing manager: does each segment warrant a distinct campaign?
Reduce wasted marketing spend by eliminating irrelevant messages sent to wrong customer groups	Percentage of customers correctly routed to relevant segment	Silhouette Score ≥ 0.35 — clusters distinct enough to route meaningfully

2.3 Proposed Solution & Final Product

An ML-based customer segmentation system built on the full MLOps lifecycle, delivering **two end products**:

Product 1 — Business Dashboard (Marketing Team) A segmentation view showing which customers belong to which segment, their RFM profile, and the recommended campaign action per segment. Updated whenever the model is retrained.

Product 2 — Monitoring Dashboard (DS Team) A Grafana-based operational dashboard showing API health, request latency, predictions per cluster over time, and input feature distributions to detect data drift.

Both are served via a REST API that returns a cluster label and business description for any given customer's R, F, M values.

3. Requirements

3.1 Functional Requirements

ID	Requirement
FR01	The system shall accept Recency, Frequency, and Monetary values as input and return a cluster label and segment description
FR02	The system shall expose a REST API endpoint POST /predict accessible over HTTP
FR03	The system shall assign every customer to exactly one segment
FR04	The system shall produce between 3 and 5 named customer segments with distinct RFM profiles
FR05	The system shall log every API request with timestamp, input values, and predicted cluster
FR06	The system shall display segment distributions and feature distributions on a monitoring dashboard
FR07	The system shall support model retraining when drift or performance degradation is detected
FR08	The business dashboard shall display each customer's segment label and recommended campaign action

3.2 Non-Functional Requirements

ID	Requirement
NF01	API response time shall be under 500ms under normal load
NF02	All ML experiments shall be reproducible — identical seed + inputs must produce identical outputs
NF03	The system shall be fully containerised and deployable via Docker on any machine
NF04	All code changes to main branch must pass automated lint, test, and build checks via CI/CD
NF05	Every function in src/ must have a docstring
NF06	The environment must be fully reproducible via requirements.txt (Python 3.12.6)
NF07	The system must be maintainable — new model versions deployable without downtime via model registry

3.3 Operational Requirements

ID	Requirement
OR01	No personally identifiable information (PII) shall be committed to the repository at any time
OR02	Every model version must be registered in MLflow Model Registry before deployment
OR03	Model promotion from Staging to Production requires a documented Go decision signed off by the full team
OR04	A retraining policy must define the conditions under which the model is retrained (CT triggers)
OR05	The monitoring dashboard must remain operational independently of the prediction API
OR06	All significant project decisions must be recorded in decision_log.md with what, why, who, when

4. Data Science Targets

4.1 Dataset

UCI Online Retail Dataset — 541,909 transactions · 8 columns · Dec 2010–Dec 2011

Column	Type	Business Meaning
InvoiceNo	String	Transaction ID — starts with 'C' if cancelled
StockCode	String	Product identifier
Description	String	Product name
Quantity	Integer	Units purchased — negative if cancelled/returned
InvoiceDate	DateTime	Date and time of transaction
UnitPrice	Float	Price per unit in GBP
CustomerID	Float	Unique customer identifier (~25% null)
Country	String	Customer country

4.2 RFM Feature Engineering

Reference date: 01 January 2012 (one day after last transaction in dataset)

Feature	Formula	Business Meaning
Recency (R)	ref_date – max(InvoiceDate) per CustomerID	How recently active? Lower = more recent
Frequency (F)	COUNT(DISTINCT InvoiceNo) per CustomerID	How often do they buy? Higher = more loyal
Monetary (M)	SUM(Quantity × UnitPrice) per CustomerID	How much have they spent? Higher = more valuable

4.3 Success Matrix

Metric	Threshold	Type	Rationale
Silhouette Score	≥ 0.35	Primary KPI	Measures cluster cohesion + separation. Below 0.35 = clusters overlap and are not usable
Davies-Bouldin Index	≤ 1.5	Primary KPI	Measures inter-cluster similarity. Above 1.5 = groups too similar to differentiate
Calinski-Harabasz Index	Maximise	Secondary KPI	Higher = denser, better-separated clusters
Number of clusters (k)	3 – 5	Business constraint	< 3 = too coarse; > 5 = operationally unmanageable for marketing team
Cluster interpretability	100%labelled	Business gate	Every cluster must have a plain-language name before Go decision
Experiment reproducibility	100%	MLOps gate	Same seed + scaler must produce identical assignments
All runs logged	Mandatory	MLOps gate	k, seed, scaler, Silhouette, DBI, CHI, inertia logged per run

4.4 Modelling Approach

Primary: K-Means clustering — Elbow Method + Silhouette Score to select optimal k (k = 2 to 10 tested) Alternative: DBSCAN or GMM — at least one alternative trained and compared across all three metrics Visualisation: PCA (2 components) for scatter plot visualisation only — not used in training Output: Cluster label (integer) + segment name + business description returned per customer

4.5 Expected Segment Structure

Segment	Recency	Frequency	Monetary	Proposed Label
Low R, High F, High M	Recent	Frequent	High spend	Champions
Low R, Medium F, Medium M	Recent	Moderate	Medium spend	Loyal Customers
Medium R, Low F, Medium M	Moderate	Infrequent	Medium spend	At Risk
High R, Low F, Low M	Long ago	Rare	Low spend	Hibernating
High R, Very Low F, Very Low M	Very long ago	Once or twice	Minimal spend	Lost Customers

Note: actual segment structure will be determined by the data. Labels assigned post-clustering based on mean R/F/M per cluster.

5. Project Plan

5.1 Phase Overview

Phase	Weeks	Owner	Key Output	Gate
Preparation	W1–2	All	Repo, requirements.txt, README, literature summary	Env reproducible on all machines
Phase 1: Business Understanding	W2–3	Kirtan	DS Project Map, requirements doc, success criteria	Full team sign-off on this document

Phase	Weeks	Owner	Key Output	Gate
Phase 2: Data & EDA	W3–6	Ayush + Jash	EDA report, data quality report, rfm_processed.csv, data pipeline	Pipeline live + feature store committed
Phase 3: Modelling	W6–10	Ayush + Jash	MLflow experiments, model assessment report, best model in Staging	All 3 metrics computed + model in Staging
Phase 4: Evaluation	W10–11	All	Final model report, Go/No-Go evaluation report	Go decision + model promoted to Production
Phase 5: Deployment	W11–13	Jash + Kirtan	FastAPI, Docker, CI/CD, business dashboard draft	API live + CI/CD passing
Phase 6: Monitoring & Utilization	W13–16	All	Grafana dashboard, drift demo, retraining policy, final report	Final report submitted W16

5.2 Branch & Collaboration Strategy

Branch Pattern	Owner	Purpose
Main	All (PRs only)	Protected — no direct pushes, always merge target
feature/<task-name>	Assigned member	One branch per pipeline or code task
document/<doc-name>	Assigned member	One branch per documentation artifact

5.3 Technology Stack

MLOps Component	Tool	Owner
Source Code Repository	GitHub	Jash
Data Versioning	DVC	Jash
Experiment Tracking	MLflow on DagsHub	Ayush + Jash
Model Registry	MLflow Registry (Staging → Production)	Jash
Serving Component	FastAPI + Uvicorn	Jash
Containerisation	Docker	Jash
CI/CD Component	GitHub Actions	Jash
Monitoring Component	Prometheus + Grafana	Jash
Business Dashboard	Grafana / lightweight frontend	Jash + Kirtan
CT Component	Retraining policy + pipeline	Ayush + Jash
Environment	Python 3.12.6 + requirements.txt	Jash

6. Repository Structure

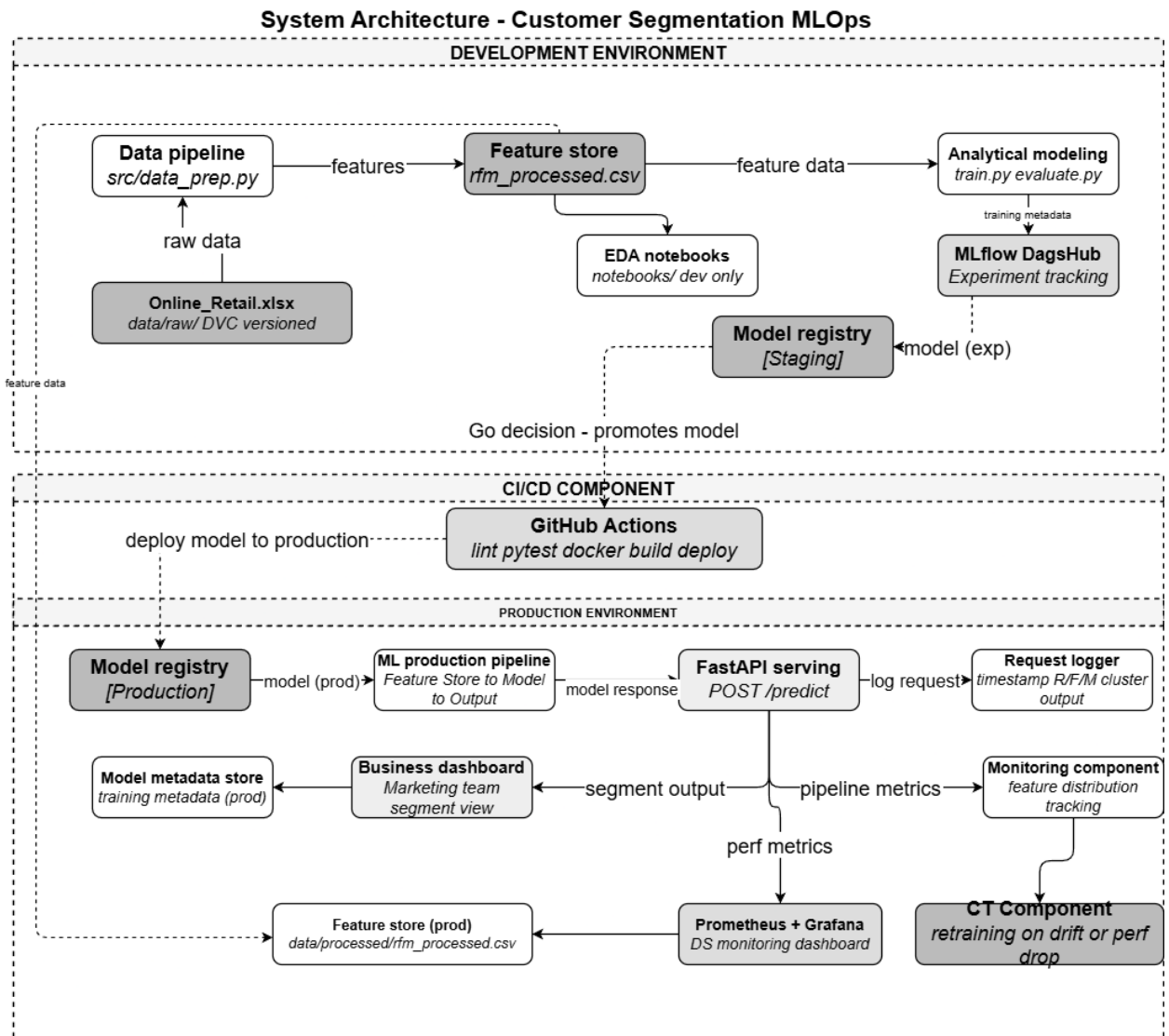
Path	Purpose
data/raw/	Raw UCI Online Retail dataset — DVC tracked
data/processed/	Feature store output (rfm_processed.csv) — DVC tracked
docs/architecture.md	System architecture documentation
docs/business_case.md	Business case and objectives
docs/reflection.md	Project reflection and lessons learned
src/data_prep.py	Data pipeline — raw data to feature store
src/train.py	Model training and MLflow experiment logging
src/evaluate.py	Cluster metrics, profiling, and business labelling
src/app.py	FastAPI serving component
src/main.py	API entry point
Dockerfile	Container definition for the serving component
dvc.yaml	DVC pipeline stages definition
requirements.txt	Pinned Python dependencies (Python 3.12.6)
Readme.md	Project overview, setup, and usage instructions

7. Data & Model Regulations

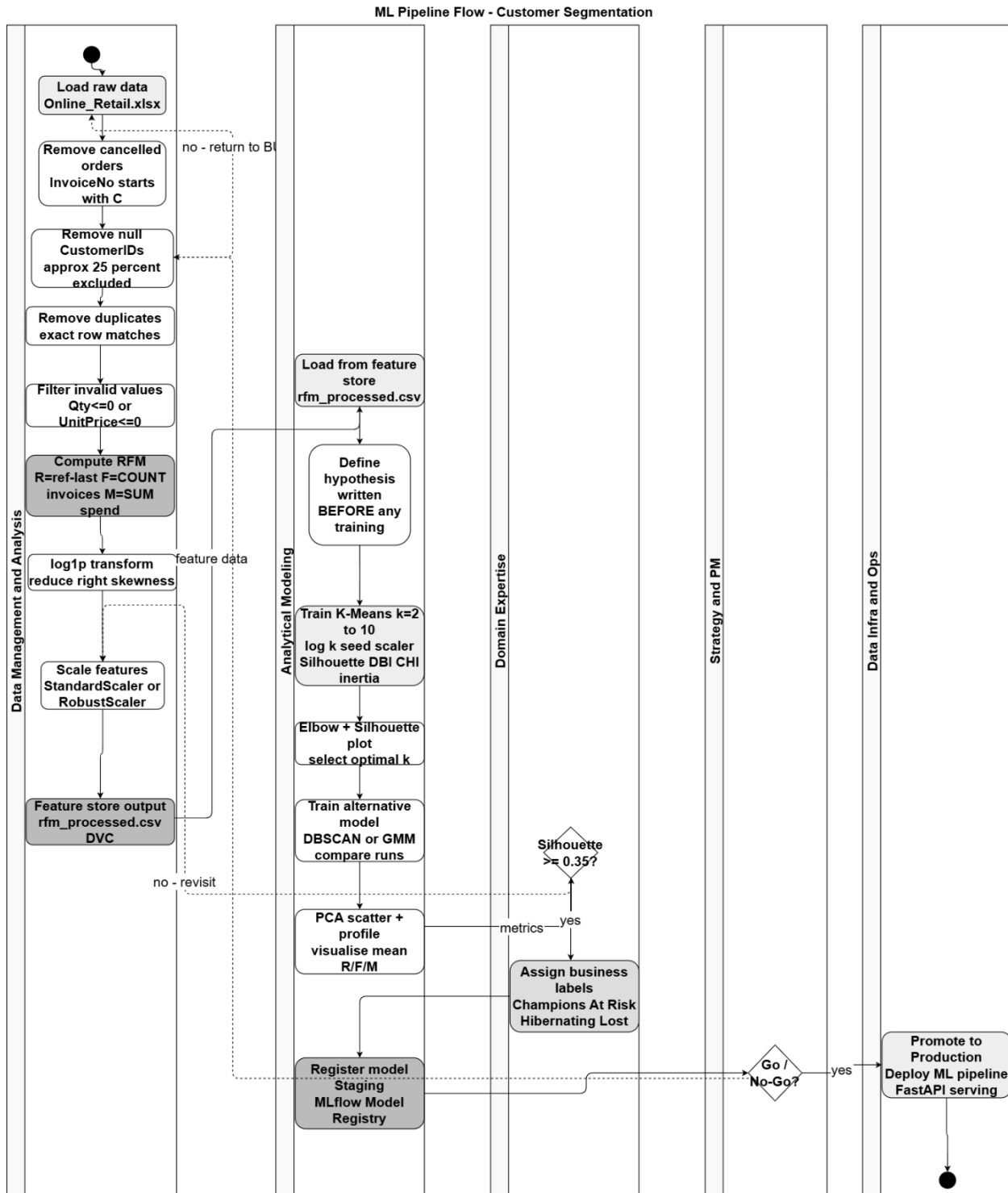
Area	Measure
GDPR	CustomerID treated as pseudonymous — no PII committed to repo at any point
Data Licence	UCI dataset is publicly available for academic use — source documented in Data Description Report
Fairness	RFM-only features — no demographic or protected characteristics used in clustering
Transparency	Cluster profiles (mean R/F/M per cluster) + business labels serve as explainability mechanism
EU AI Act	Use case is marketing personalisation only — non-prohibited, low-risk classification
Model Versioning	No model may be deployed to Production without being registered and approved in MLflow Registry

8. Architectural Overview

8.1 System Architecture (Three-Layer)



8.2 ML Pipeline Flow



8.3 CI/CD Pipeline

CI/CD Pipeline - GitHub Actions

